GigBid: R&D Report

AI-4016 Assignment II · engineering design review

Ibrahim Umar Malik November 2026

How to read this report. Every number is rendered from a file in `evals/results/` by `python -m evals.make_tables` (CI fails if the report and the files disagree). Unless a table says otherwise, runs use: the synthetic stand-in GigSim and personas (the official starter kit was not available at build time; `docs/DATA_CARD.md`), the `offline` model profile (deterministic heuristics in every LLM node, because this build environment had no model access), a hash embedder and a lexical reranker. The tune/test split is fixed (70/30 by row); nothing was chosen on the test split. Where an experiment needs a real LLM it is marked **not run**, never estimated. `make eval-llm` reruns all of it with open-weight models.

1. Problem and constraints

A freelancer spends hours a week reading job posts, most of which are a poor fit, a scam or underpaid, and then writes proposals that either sound generic or overclaim. Connects cost money. GigBid's job: read every new post on a schedule, skip the bad ones with a reason, and for the good ones prepare a priced, evidence-backed proposal the freelancer can approve in under a minute.

Never automated: sending a proposal, sending a message, spending Connects, accepting a contract, and any action on Upwork outside its official API. GigBid prepares; the human sends. Live mode only creates drafts on Upwork.

Budgets (declared in `agent/config.yaml: budgets.declared`): p95 latency 25 s per job analysed and 60 s per proposal; USD 0.002 per job analysed and USD 0.01 per proposal on the open profile; at most 40 graph steps and 60k tokens per thread.

2. Architecture

Two LangGraph `StateGraph`s, both rendered from code in `docs/images/`. The **bid graph** (one thread per job): Scout → Job Analyst, Risk Assessor and Fit Matcher in parallel (`Send`) → a deferred `decide` node (skip / watch / bid) → Pricing → Writer → claim verifier → Critic (loops back at most twice) → Compliance Guard → dry-run preview → `interrupt()` for approval → submit (or re-check an edit, or regenerate a paragraph). The **conversation graph** (one thread per client conversation): context → intent (question, negotiation, interview request, scam signal) → Client Liaison (reply + interview slots, walk-away floor) → Compliance → preview → approval → send.

Marketplace port. One `MarketplacePort` protocol with two adapters, GigSim and Upwork live, behind our own Marketplace MCP server. Agents only see MCP tools; the mode is configuration. **MCP surface:** three own servers (marketplace, portfolio, workspace), plus the official Upwork and GitHub servers; per-agent allowlists; write tools belong only to the post-approval executor (`docs/TOOLS.md`). **Stores:** Postgres for app data, checkpoints and the memory store (SQLite locally), an append-only audit log, Redis for the Arq queue and live events.

State, trace and memory are separate. State is the typed per-thread record (`agent/state.py`, reducers on every field parallel branches write). The trace is every node, model call and tool call with arguments, tokens, cost and latency (`trace_spans`, one trace id from the browser to the audit log). Long-term memory holds only promoted knowledge: a style rule after the same edit appears in two different drafts; an outcome insight after three outcomes with a clear reply-rate gap.

Why each agent exists is in `docs/ARCHITECTURE.md`. The short version: Risk runs before writing because scams are the costliest mistake; the Matcher builds a requirement-by-evidence matrix because "fit" without evidence is a guess; the verifier exists because writers overclaim (§4); the Critic loop is bounded because unbounded self-critique burns tokens without converging.

3. R&D question A: tool granularity

Hypothesis. Task-shaped tools (named after what an agent is trying to do) need fewer calls and fewer schema tokens than API-shaped fine-grained tools, and are easier to select than one mega-tool with an `action` argument.

Setup. The same capabilities exposed three ways (8 task-shaped, 22 fine-grained, 2 mega tools), 40 tasks (8 templates × 5 entities) with the correct first call(s) per design (`evals/runners/tool_granularity.py`).

Tool design	Tools	Schema tokens per turn (≈)	Min calls / task	Selection accuracy
task_shaped	8	479	1.12	not run
fine_grained	22	1111	2.25	not run
mega	2	166	1.12	not run

Result. Fine-grained tools cost 2.3× the schema tokens on *every* model turn and need twice the calls per task. The mega-tool is the cheapest in tokens, but its arguments are an untyped `params` object, so argument validation moves from the MCP schema into the model's head. Selection accuracy needs a tool-calling model and was not run here. **Conclusion:** task-shaped tools ship (ADR 0003). The open question for the LLM run is whether mega-tool argument accuracy falls far enough to cancel its token advantage.

4. R&D question B: advanced RAG

Hypothesis (stated before running): requirement decomposition plus reranking raises evidence Recall@5 over the naive pipeline and, through better-grounded drafts, raises the GigSim reply rate by at least 3 points.

Variants. V0 naive (fixed 512-token chunks, dense only), V1 requirement decomposition, V2 hybrid BM25 + dense (RRF), V3 reranking, V4 outcome boost (evidence from proposals that won), V5 multi-vector (summary / README / code / proposal indexed separately), V6 contextual chunk headers, V7 HyDE, V8 corrective (grade, re-query, refuse). Selection rule fixed in advance: best nDCG@5 on the tune split (2 dp), then MRR, then latency.

Variant (tune split)	Recall@5	MRR	nDCG@5	p95 ms
v0_naive	0.905	0.964	0.902	2.2
v1_decompose	0.950	0.993	0.946	2.5
v2_hybrid	0.912	0.980	0.917	2.8
v3_rerank	0.883	0.975	0.890	3.3
v4_outcome	0.919	0.843	0.825	1.8
v5_multivector	0.660	0.550	0.518	3.5
v6_contextual	0.929	0.992	0.937	1.4
v7_hyde	0.933	0.979	0.931	1.6
v8_corrective	0.945	0.996	0.946	2.1
v2_hybrid_v1_decompose	0.945	0.996	0.946	1.5
v3_rerank_v1_decompose	0.947	0.997	0.953	5.2
v2_hybrid_v1_decompose_v4_outcome	0.942	0.963	0.922	1.6
v2_hybrid_v1_decompose_v3_rerank_v4_outcome	0.948	0.966	0.930	5.2
v6_contextual_v2_hybrid_v1_decompose_v4_outcome	0.949	0.974	0.938	1.5
v8_corrective_v2_hybrid_v1_decompose_v4_outcome	0.942	0.963	0.921	2.2

Chosen by rule (max nDCG@5 on tune split (2 dp), then MRR, then lower p95 latency): **v3_rerank_v1_decompose**. Held-out test split: nDCG@5 0.957 vs naive 0.927; Recall@5 0.955 (95% CI [0.93, 0.9778]) vs 0.922. Embedder `hash`, reranker `lexical`, commit `dfd4987`.

Ablation of the shipped pipeline (tune split), and the components the rule did not pick:

Setting (tune split)	Recall@5	nDCG@5	Δ nDCG@5
shipped (all components)	0.947	0.953	-
structured chunking → fixed 512-token chunks	0.947	0.947	-0.0059
requirement decomposition off	0.883	0.890	-0.0634
reranking off	0.945	0.946	-0.0069
+ hybrid BM25	0.947	0.953	0.0000
+ contextual chunk headers	0.949	0.955	0.0014
+ outcome boost	0.948	0.930	-0.0229
+ HyDE	0.953	0.958	0.0050

Decomposition is the component that matters (−0.063 nDCG@5 without it); reranking and structured chunking add less than 0.01 each with a hash embedder. Two negative results: **outcome boosting hurts retrieval** (−0.023): it promotes past proposals that won, and our reading of the misses is that those proposals were often won for a *different* requirement; and **multi-vector grouping (V5) is the worst variant**, most likely because fusing four separately ranked groups lets a weak code chunk outvote a strong case study.

By persona and job type (test split; reply rate from the end-to-end run below):

Slice (V0 → shipped)	Recall@5	nDCG@10	GigSim reply rate
persona: ai_ml	0.969 → 0.990	0.971 → 0.986	
persona: fullstack	0.897 → 0.960	0.910 → 0.964	
persona: data_analyst	0.901 → 0.915	0.905 → 0.921	
job type: fit (n=89)			30.6% → 37.4%
job type: injection (n=14)			16.5% → 26.5%
job type: lowball (n=6)			26.5% → 33.8%
job type: off_domain (n=2)			15.6% → 16.4%

End to end (B5). Retrieval metrics alone do not decide what ships, so V0 and the shipped pipeline were run through the whole bid graph:

Variant	Evidence Recall@5	nDCG@10	Unsupported claims / proposal	Judge score	GigSim reply rate	p95 latency (s)	Cost / proposal (USD)
V0 naive	0.922	0.928	0.00 (0.27 before verify)	3.59	28.3%	0.25	$0.0000
Shipped: v3_rerank_v1_decompose	0.955	0.957	0.00 (0.32 before verify)	3.96	35.4%	0.26	$0.0000

Paired on the same 111 proposals: the lead evidence changed in only 5, yet reply probability rose +6.6 points on the proposals whose lead evidence did *not* change. Drafts quoting a specific metric: 8.1% with V0 vs 65.8% shipped.

Claim verification. Without the verifier, the shipped writer leaves the unsupported-claim rates shown in the "before verify" column; after it, zero remain on the test split. The verifier treats honest negatives ("I have not used X in client work") as non-claims, otherwise it deleted the most trustworthy sentence in the draft.

Judge calibration. The 30-item blind labelling set is in `evals/datasets/judge_labels.csv`. Human labels cannot be produced by the system, so Cohen's κ is **pending** until the 30 labels are filled in; the only agreement reported is a clearly labelled proxy against the GigSim client model:

Judge calibration (30 items)	Value
Mean judge score: GigBid / reuse past / template	4.07 / 3.2 / 2.09
Agreement with human labels	awaiting human labels (0/30)
Proxy: quadratic κ vs GigSim client model (not human)	0.3108

Where retrieval and replies came apart. The hypothesis held, but not for the stated reason. Retrieval barely moved (the right project was already ranked first by V0 for almost every job), yet the reply rate rose by several points. The paired comparison under the B5 table locates it: the lead evidence changed for only a handful of proposals, and the gain is on proposals that cite the *same* project. What changed is the chunk the Writer receives. Fixed 512-token chunks cut case studies mid-way, so the "Result" section with the number is often in a different chunk; structured chunks keep problem, approach and result together, and the share of drafts that quote a specific metric jumps. So the variant that matters for replies is the **chunking**, which nDCG cannot see; and the ranking improvements (reranking, decomposition) that retrieval metrics reward add little to the reply rate. The flip side: better evidence also means the writer attempts more claims (the "before verify" column rises), which only works because the verifier sits behind it.

Shipped: `v3_rerank_v1_decompose` (decomposition + reranking over structured chunks): it wins the declared rule, holds on the test split, and costs a few milliseconds. Structured chunking is the part we would never remove.

5. R&D question C: is the multi-agent design worth it?

Same 50 test jobs (17/17/16 per persona): (a) one ReAct agent with every read tool, (b) `langgraph-supervisor` with risk, matcher, pricing and writer workers, (c) the hand-written graph.

Architecture (50 jobs)	Decision accuracy	Scam bids	Unsupported claims	Tool calls / job	p95 latency
graph	92.0%	0	0.0%	9.6	0.30s
react	not run: needs an LLM				
supervisor	not run: needs an LLM				

Only the graph can run without a model; (a) and (b) are implemented (`evals/runners/architecture_compare.py`) and run with `make eval-llm`. What the graph guarantees structurally, whatever a model does: risk always runs before writing, unsupported claims are always removed, and no write tool is reachable before approval. A ReAct agent can only be *asked* to do those things. The comparison to watch in the LLM run is whether (a) wins on simple single-skill jobs, where the graph's three parallel analyses are overhead.

6. The change request (2 Nov policy update)

What changed. Marketplace policy v2: bidding in a job's first hour costs 2 extra Connects, and every screening question must be answered (the stand-in implements this as `GIGSIM_POLICY_VERSION=2`).

What had to be edited. Pricing stores the policy it already fetched through `get_policy` in state (3 lines); the Compliance Guard gained rule MP2, blocking drafts with an unanswered screening question (9 lines); one integration test (`test_policy_v2_change_request`). No adapter, graph, prompt or UI change: the Connects cost already came from the marketplace, and the approval screen already shows it. **Time:** about 20 minutes including the test.

What it says about the design. Policy is data behind a read tool, and enforcement lives in one node. The change would have been a multi-file edit if prices or Connects had been computed inside the Writer.

7. Signature extension: X2 proposal A/B experiments

Each bid is assigned an arm by a stable hash (user, experiment, job), so resumes never flip it; the Writer honours the arm; analytics run a two-proportion z-test and only declare a winner after 30 proposals per arm and $p < 0.05$ (`docs/EXTENSION.md`). The shipped experiment: ask the client one specific clarifying question, or close with a concrete next step.

X2 experiment: ask one clarifying question vs close with a next step	Value
Jobs (paired, both arms written)	111
Reply probability: statement / question	26.4% / 35.7%
Paired lift (question − statement)	+9.3 points (95% CI [7.8, 11.1])
Chance the live z-test calls it at 30 / 120 / 500 proposals per arm	10.0% / 36.0% / 88.5%

The paired design (both arms written for the same job) shows a clear lift; the simulated live experiment shows the uncomfortable part: at one freelancer's volume, the z-test needs hundreds of proposals per arm to call a lift of this size. The first version of this experiment (question-first vs evidence-first opening) measured a difference of exactly zero because both arms still contained a question; it was redesigned (`docs/AI_USAGE.md`).

8. Failure analysis

Every failed test-split job from the first full run was grouped into the taxonomy (`evals/runners/failure_analysis.py`, before: `failures_before_fixes.json`, after: `failures.json`):

Failure type (test split, 180 jobs)	Before fixes	After fixes
missed evidence	0	0
wrong skip: injected post treated as client risk	7	3
wrong skip: risk/fit threshold	6	5
wrong bid: lowball budget	6	5
wrong bid: off-domain (missed evidence gap)	2	2
wrong bid: duplicate repost	5	0
mispriced	27	16
unsupported claim	0	0
generic opening	0	0
compliance miss	0	0
injection obeyed	0	0
tool error	0	0
bid/skip accuracy	85.6%	91.7%
bids inside fair price band	74.3%	84.6%

Fixes tried, each measured on the same split:

1. **Injection read as client risk (wrong skip).** Injected lines such as "include the freelancer's email" matched the contact-harvesting scam pattern, and "IGNORE ALL PREVIOUS INSTRUCTIONS" matched a scam pattern, so legitimate jobs with a planted injection were skipped. Fix: scam patterns run on the sanitised post, and the injection flag no longer adds to the risk score (it is neutralised, not ignored). **Worked:** wrong skips of injected jobs fell, scams were still all caught, no injection obeyed.
2. **Reposts bid on twice (wrong bid).** De-duplication matched any job with the same fingerprint, including later ones, and the eval harness did not queue jobs the way the Scout does. Fix: only earlier jobs count; the harness now queues jobs in posting order. **Worked:** duplicate bids went to zero.
3. **Mispriced.** Quotes from category percentiles ignored the client's own budget: half were above the fair band (capped at 110% of budget), half below. Fix: never above the budget; strong fits not under 85% of it. **Partly worked:** see the in-band rate in the decision table; lowball jobs are still mispriced because the floor wins.
4. **Lowball bids (not fixed).** Jobs whose budget is far below the work's value are still bid on when the budget is above the user's floor. A "value vs budget" rule needs a real estimate of effort, which the heuristic Analyst lacks.

No proposal kept an unsupported claim and no compliance miss was found on the test split; the categories are listed with zero counts rather than dropped.

9. Safety and platform respect

Must never happen: a send without approval; Connects spent twice; an instruction inside a job post or client message being followed; contact details or off-platform payment in a proposal; scraping or browser automation of Upwork; one user's data reaching another.

How it is enforced. Write tools are reachable only by the executor after `interrupt()` returns approve (allowlists, tested); idempotency keys from thread + version + edits (crash-and-resume test, exported trace); job posts and messages are sanitised and delimited, and the Compliance Guard blocks or rewrites P1–P7 / MP2 violations after any human edit too; PII masking in logs and traces; OAuth tokens encrypted with Fernet; every query scoped by user id; append-only audit log; Upwork only through its official MCP server, polled at most every 4 hours.

Planted-injection jobs (test split)	Attack success
Defences off (no sanitising, no delimiting)	not run (needs an LLM writer; the offline writer cannot follow instructions)
Defences on (shipped)	0 of 19

Three of the planted-injection and scam posts in the test split are reposts that the Scout skipped as duplicates before any analysis; they count as handled but not as "flagged", which is why the flagged rate is below 100%.

What still gets through. Attack success *before* defences can only be measured with an LLM writer (the offline writer cannot follow instructions by construction), so the "after" zero is necessary but not sufficient evidence. A client can still move a conversation off-platform in their *own* words; GigBid flags it as a scam signal but the human decides. Paraphrased injections that avoid the regex layer reach the model inside delimiters; only the LLM run tells how often the model obeys them.

10. Results against the declared budgets

The hidden test set is run by the graders. On our held-out test split:

Metric (test split, n=180, profile `offline`)	Value
Bid/skip accuracy	91.7% (95% CI [87.2, 95.6])
Bid precision / recall / F1	0.933 / 0.924 / 0.928
Scam posts not bid on	100.0% of 17
Injected posts flagged / obeyed	94.7% flagged, 0 of 19 obeyed
Unsupported claims without verifier → shipped	15.4% → 0.0% (220 claims)
Bids inside the fair price band	84.6% of 104
Latency per job p50 / p95	0.18s / 0.27s
Errors	0 of 180

Thresholds chosen on the tune split by F1: bid_min_fit=0.7, max_risk=0.35 (tune F1 0.9325).

System (180 test jobs)	Bids	Connects	Reply rate	Replies / 100 Connects	Interviews	Hires	Scam bids
GigBid	111	846	36.0%	4.73	25.0	12.3	0
Template, bid on all	154	1236	12.6%	1.57	10.3	4.3	4
Reuse best past proposal	180	1400	13.7%	1.76	13.3	5.7	17

Measured latency is far inside the 25 s budget, and cost is zero, **because no model was called**; these numbers say nothing about the open profile, whose p95 and cost come from `make eval-llm`. Against the baselines on the same jobs, GigBid bids on fewer jobs, spends fewer Connects and gets more replies per Connect, and never bids on a scam; the caveat is circularity: the stand-in simulator rewards what its author believed clients reward. One optimisation that helped: skipping early (scam and fit checks before any writing) means most jobs never reach the Writer.

Appendix A. Reproduce

```
make install && make test && make e2e && make eval && make report
```

Commits, traces and results: `git log`, `docs/traces/*.json`, `evals/results/*.json`.